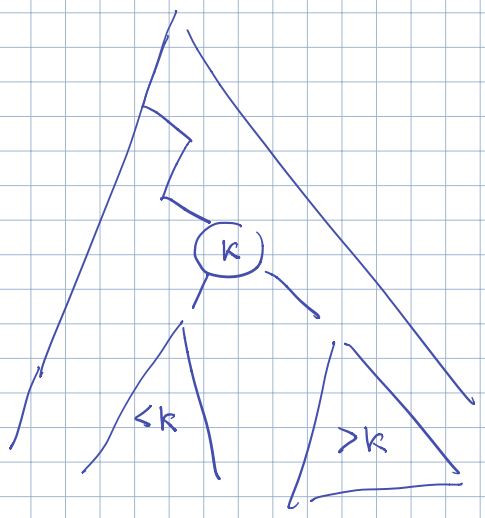
BST

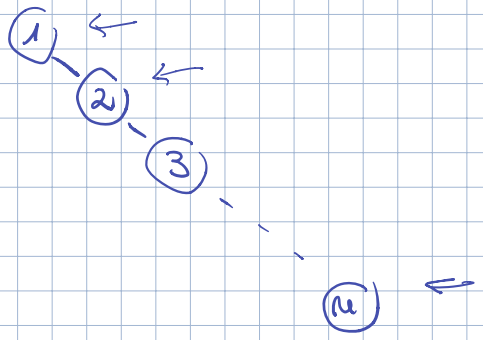


T con n chiavi
h altezza di T

$h \in \Omega(\log n)$

$h \in O(n)$

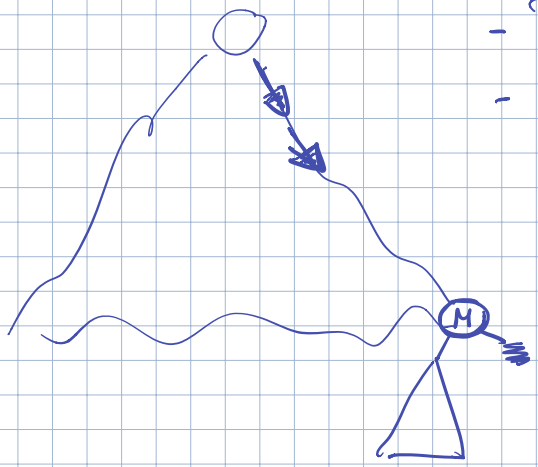
Esempio



T BST sse InOrder(T) restituisce le chiavi in ordine crescente

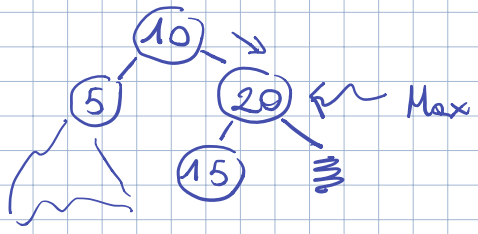
- Ricerca
- Inserim.
- Cancell.

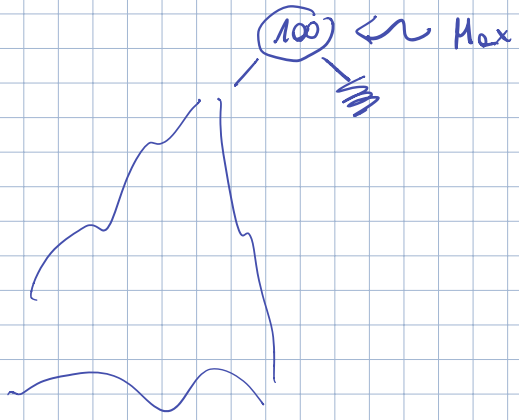
Ricerca Max



- Parto dalla radice
- Mi fermo quando arrivo a un nodo che non ha figlio dx

Esempio





```
BST_Max ( x ) {  % x ≠ NIL        O(h)        O(n)
    if ( x.right = NIL ) {
        return x
    } else {
        return BST_Max ( x.right )
    }
}
```

Main

```
... if (T.root ≠ NIL)
        BST_Max (T.root)
    else return NIL
```

```
BST_max_while (T) {                O(h)    O(n)
    x ← T.root                              In-Place
    if (x = NIL) return x
    while (x.right ≠ NIL) {
        x ← x.right
    }
    return x
}
```

Scrivere ricerca minimo

Ricerca di k (chiave)

- Parto dalla radice
- Arrivato ad x
  - se x.key = k ho finito $k \in T$
  - se x.key > k scendo a sx
  - se x.key < k scendo a dx
  - se x = NIL ho finito $k \notin T$

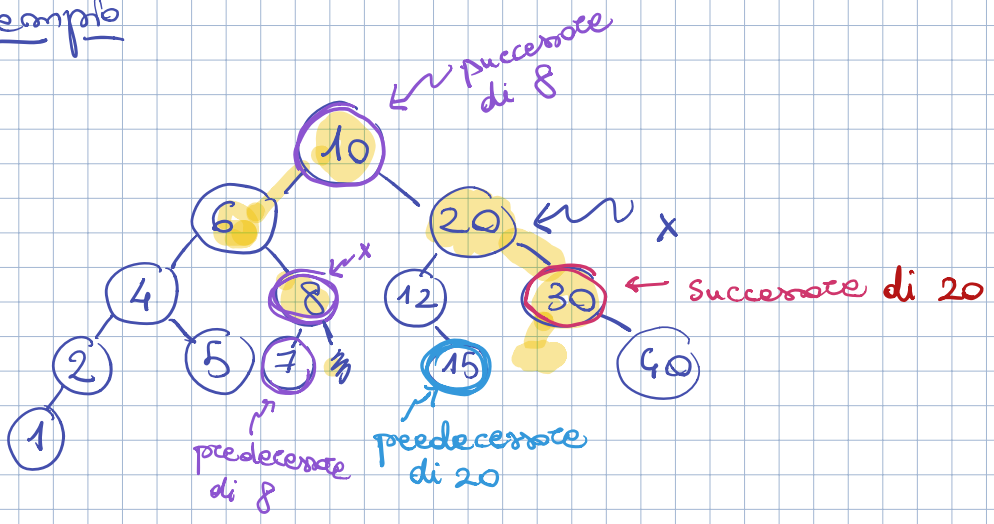
```
BST-Search (x, k) {
    if (x = NIL || x.key = k) {
        return x
    } else {
        if (x.key > k) {
            return BST-Search (x.left, k)
        } else {
            return BST-Search (x.right, k)
        }
    }
}
Main                BST-Search (T.root, k)
```

Ricerca successore (predecessore)

Dato T BST x nodo di T

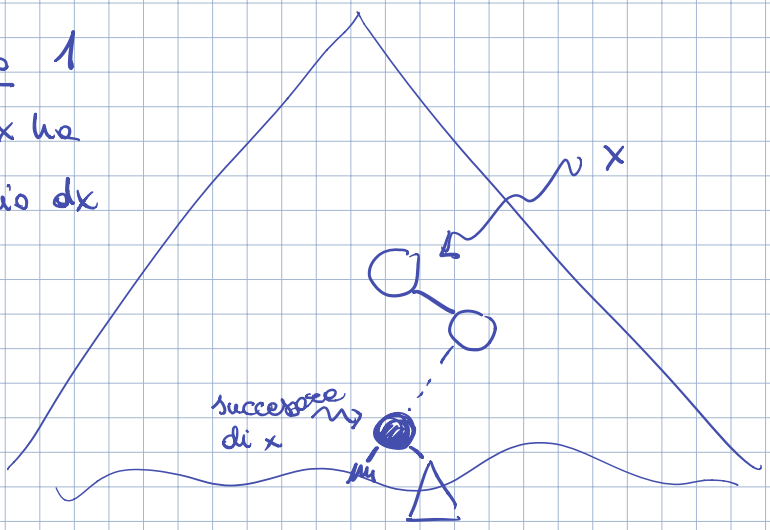
Trovare y che contiene la più piccola chiave più grande delle chiave di x

Esempio



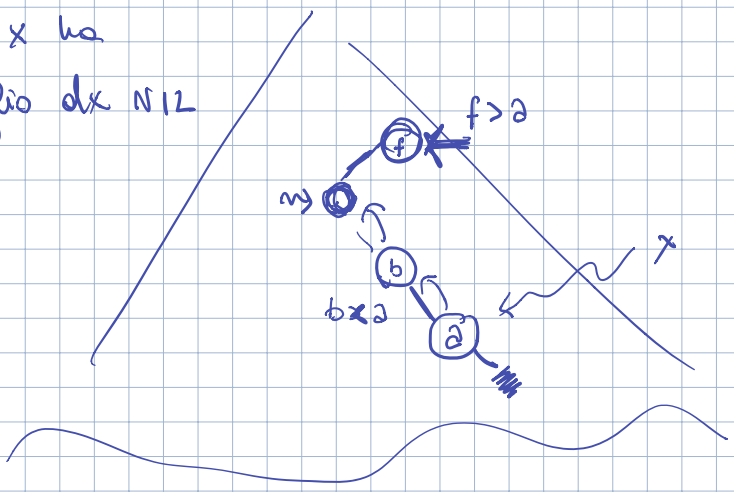
Caso 1

se x ha figlio dx



Caso 2

se x ha figlio dx NIL



Mi fermo quando arrivo da sx

Se x è il Max arrivo al genitore della radice che è NIL e mi fermo

```
BST_successor (x) {   % x ≠ NIL   O(h)  O(n)
    if (x.right ≠ NIL) {
        return BST-MIN (x.right)        in-place
    } else {
        y ← x.parent
        while (y ≠ NIL && x = y.right) {
            x ← y
            y ← y.parent
        }
```

return y

}

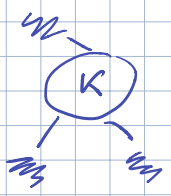
}

Scrivere Predecessore Riscrivere InOrder inplace

~ . ~ . ~ .

Inserimento in un BST

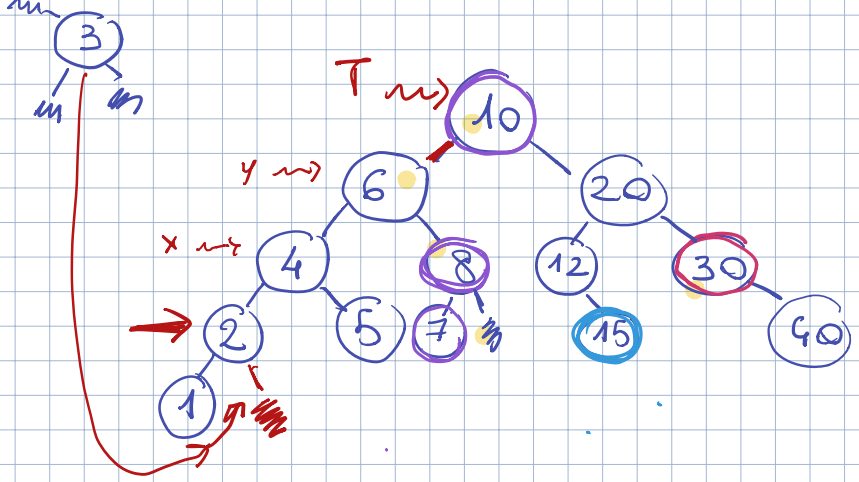
T k memorizzata in z modo



z

Inserire z in T

Esempio



Idea

- Scendo nell'albero come se cercassi z.key
- Uso 2 puntatori x e y. In ogni istante y è il parent di x
- Quando x arriva a NIL aggancio z ad y

All'inizio $x = T.root$ e $y = NIL$

Se $T.root = NIL \Rightarrow z$ è la nuova radice

Esempio

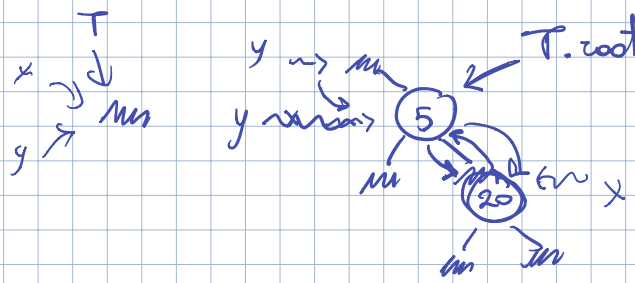
```
BST_Insert (T, z) {   % z ha chiave K
                      % parent e figli NIL
    y ← NIL
    x ← T.root
    while (x ≠ NIL) {
        y ← x
        if (x.key > z.key) {
            x ← x.left
        } else {
            x ← x.right
        }
    }
    z.parent ← y
    if (y = NIL) {
        T.root ← z
    } else {
        if (y.key > z.key) {
            y.left ← z
        } else {
            y.right ← z
        }
    }
}
```

O(h)
O(n)

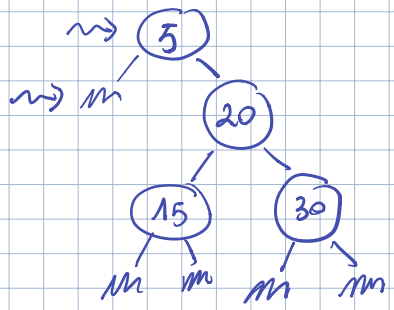
Esempio

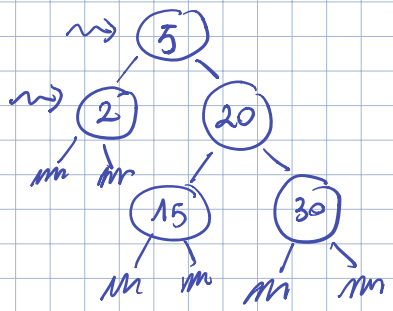
Partire da T vuoto inserire le chiavi

5 20 30 15 2 10 12 18 40

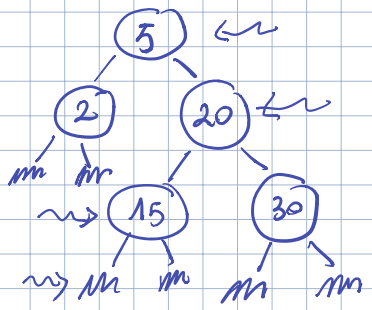


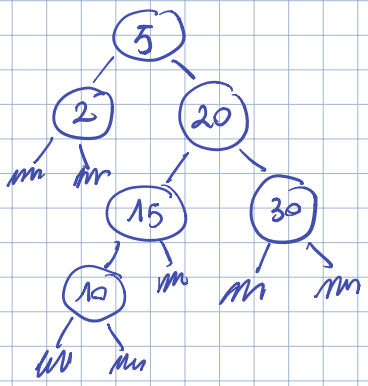
Inserisco 2



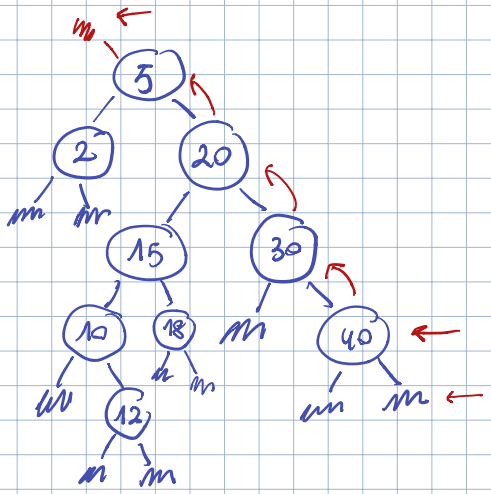


Inserisco 10





Inserisco
12, 18, 40



Ricerca successore 40